Node.js Fresher 01 - Khái niệm cốt lõi

Mục	Nội dung
Cấp độ	Fresher
Module	A1 - Khái niệm cốt lõi của Node.js (Unit 1, Session 1)
Chủ đề	Tổng quan và thuật ngữ, Cài đặt môi trường, Kiến trúc (Event Loop, V8 Engine, Non-blocking I/O), NPM (package.json, install, scripts), Core modules: path và fs (xử lý file)
Mục tiêu	Hiểu Node.js là gì và chạy code như thế nào (V8 + libuv + Event Loop), thiết lập một dự án TypeScript, quản lý dự án bằng npm, và đọc/ghi file an toàn với hai core module path và fs.
Yêu cầu trước	Biết cú pháp cơ bản của JavaScript/TypeScript (biến, hàm, object, mảng); biết dùng terminal ở mức cơ bản (cd, ls/dir, mkdir).
Phiên bản giả định	Bản LTS hiện hành của Node.js, bản ổn định hiện hành của TypeScript, ES Modules ("type": "module"). Hãy tham khảo lịch phát hành chính thức để chọn phiên bản mà nhóm của bạn đang dùng.

0. Khởi động - Chương trình Node.js đầu tiên

Nếu bạn chưa từng chạy Node.js, hãy làm 3 bước sau trước khi đọc tiếp.

a. Cài đặt tối thiểu

1. Node.js bản LTS - tải từ nodejs.org (miễn phí) hoặc cài qua một công cụ quản lý phiên bản như nvm.
2. Trình soạn thảo - Visual Studio Code (miễn phí) cùng extension ESLint.

Kiểm tra đã cài thành công:

```
node -v
# -> in ra phiên bản Node.js đã cài
npm -v
# -> in ra phiên bản npm đã cài
```

b. Chương trình nhỏ nhất

Tạo file hello.ts:

```
const name: string = "Node";
console.log(`Hello, ${name}!`);
```

Chạy file (tsx chạy trực tiếp TypeScript, không cần bước build riêng):

```
npx tsx hello.ts
# -> Hello, Node!
```

c. Boilerplate - những "bùa chú" tạm thời

Mọi dự án trong khóa học này đều bắt đầu bằng 3 lệnh giống nhau. Bạn chưa cần hiểu hết; chúng sẽ được giải thích ở mục 4.2 và 4.4.

```
npm init -y
npm i -D typescript tsx @types/node
npm pkg set type=module
```

Bạn chưa cần hiểu hết mọi thứ ngay. Cứ viết, chạy được, thấy kết quả - rồi mọi thứ sẽ rõ dần.

1. Bối cảnh vấn đề (Problem Context)

Neo (Anchor)

Bạn muốn xây một công cụ nhỏ tên NoteKeeper cho nhóm. Người dùng có thể lưu ghi chú ngắn, xem danh sách và đọc lại. Sau đó nhóm còn muốn xem ghi chú qua một trang web nhỏ. Công cụ cần đơn giản, chạy được trên mọi máy (Windows, macOS, Linux) và không đòi hỏi cài đặt nặng nề.

Cách làm thủ công (Manual)

Cách "tự nhiên" nhất của người mới: viết một script, đọc file bằng các lệnh đồng bộ, ghép đường dẫn bằng cách nối chuỗi, và tự chép file thư viện vào thư mục dự án.

```
// cách làm ngây thơ
const dir = "C:\\notes\\";              // đường dẫn Windows cố định
const file = dir + title + ".txt";      // nối chuỗi
const text = readFileSync(file, "utf8"); // chờ đến khi ổ đĩa trả lời
```

Điểm gãy (Breaking point)

Công cụ được nhiều người dùng. Khi đó:

- Trang web bị đứng với mọi người trong lúc một ghi chú lớn đang được đọc từ đĩa.
- Một đồng nghiệp dùng macOS gặp lỗi "file not found" vì đường dẫn dùng dấu gạch chéo ngược.
- Một đồng nghiệp khác chép sang phiên bản thư viện khác - chạy được trên máy này nhưng lỗi trên máy kia ("works on my machine").

Vấn đề cốt lõi: chương trình ngồi chờ vô ích trước các thao tác chậm (đĩa, mạng); đường dẫn và thư viện phụ thuộc đều quản lý bằng tay; không có gì được chuẩn hóa giữa các máy.

Cầu nối (Bridge)

Ta cần một môi trường chạy (runtime) mà (1) không ngồi chờ khi đang I/O, (2) có sẵn module chuẩn xử lý đường dẫn và file chạy được trên mọi hệ điều hành, và (3) có trình quản lý gói để khóa phiên bản thư viện. Đó chính là Node.js, các core module của nó và npm.

2. Cách tiếp cận cũ (Old Approach - trước/sau từng bước)

Một web server đồng bộ với đường dẫn cố định, viết theo cách "tự nhiên":

```
import http from "node:http";
import { readFileSync } from "node:fs";

http
  .createServer((req, res) => {
    const file = "C:\\notes\\" + req.url + ".txt"; // phụ thuộc OS + không an toàn
    const text = readFileSync(file, "utf8");        // chặn toàn bộ server
    res.end(text);
  })
  .listen(3000);
```

Vấn đề:

- Blocking: trong lúc readFileSync chờ đĩa, không request nào khác được phục vụ.
- Không portable: đường dẫn chỉ chạy trên Windows và chỉ trên một máy cụ thể.
- Không an toàn: req.url do người dùng gửi lên; một giá trị như ../../secret cho phép họ đọc file ngoài ý muốn (path traversal).
- Không xử lý lỗi: file không tồn tại sẽ ném exception và làm sập cả server.
- Không quản lý phụ thuộc: không có gì ghi rõ cần thư viện nào hay phiên bản Node.js nào.

3. Điểm yếu (Weakness)

3.1 Lệnh blocking - một việc chậm làm đứng tất cả

```
const a = readFileSync("big-1.txt", "utf8"); // 200 ms
const b = readFileSync("big-2.txt", "utf8"); // 200 ms
// 400 ms mà luồng chính không làm được việc gì khác
```

Node.js chạy JavaScript của bạn trên một luồng chính. Nếu luồng đó phải chờ, mọi người dùng khác cũng phải chờ.

3.2 Đường dẫn tự ghép - hỏng khi đổi hệ điều hành

```
const f1 = "data\\notes.txt";   // chỉ chạy trên Windows
const f2 = "data/" + name;      // thành hai dấu // nếu name bắt đầu bằng "/"
```

3.3 Phụ thuộc không được quản lý - "works on my machine"

Không có package.json và file lock, mỗi lập trình viên cài phiên bản tùy ý. Lỗi chỉ xuất hiện trên một số máy và rất khó tái hiện.

3.4 Dữ liệu đầu vào không được kiểm tra khi đưa vào đường dẫn

Nối dữ liệu người dùng vào đường dẫn mà không kiểm tra có thể cho phép đọc hoặc ghi đè file nằm ngoài thư mục dự kiến.

4. Lý thuyết - Các khái niệm cốt lõi của Node.js

4.1 Tổng quan và thuật ngữ

Node.js là một JavaScript runtime mã nguồn mở, đa nền tảng, xây dựng trên engine JavaScript V8. Nó cho phép chạy JavaScript bên ngoài trình duyệt: web server, công cụ dòng lệnh, script, công cụ build.

Thuật ngữ	Ý nghĩa	Ví dụ
Runtime	Môi trường thực thi code và cung cấp các API	Node.js, Deno, trình duyệt
V8	Engine biên dịch JavaScript thành mã máy	Cũng được dùng trong các trình duyệt nền Chromium
libuv	Thư viện C đứng sau Node.js, thực hiện I/O bất đồng bộ và chạy Event Loop	File, mạng, timer
Event Loop	Vòng lặp lấy các tác vụ đã xong và chạy callback của chúng	Callback của setTimeout
Non-blocking I/O	Khởi động một thao tác I/O rồi làm tiếp việc khác; được báo khi nó xong	fs.promises.readFile
Module	Một file hoặc gói xuất ra code có thể tái sử dụng	node:fs, ./notes.ts
Core module	Module có sẵn trong Node.js, không cần cài	node:path, node:fs
npm	Trình quản lý gói của Node.js và kho gói công khai	npm install <tên gói>
package.json	File mô tả dự án, các phụ thuộc và scripts	Xem mục 4.4
TypeScript	Tập mở rộng của JavaScript có kiểu, được biên dịch ra JavaScript thuần	const n: number = 1

Vòng đời của một tiến trình Node.js

```
[node app.js] -> [nạp + chạy module chính] -> [Event Loop khi còn việc chờ] -> [thoát]
      |                    |                                |                        |
  khởi động V8   code cấp cao nhất chạy       timer, I/O, promise chạy       hết việc,
                 một lần, từ trên xuống       khi chúng sẵn sàng             tiến trình
kết thúc
```

Bên dưới - Node.js gồm những gì

- Code chạy ở đâu: V8 thực thi JavaScript của bạn trên một luồng chính, dùng call stack (các lời gọi hàm) và heap (các object).
- Ai làm việc chờ: libuv nhờ hệ điều hành theo dõi socket và timer (epoll trên Linux, kqueue trên macOS, IOCP trên Windows). Thao tác file system và một số việc crypto/DNS chạy trên thread pool của libuv (một số luồng nhỏ, có thể cấu hình).
- Thứ tự được đảm bảo: các callback của bạn luôn chạy lần lượt trên luồng chính, nên hai hàm JavaScript không bao giờ chạy đúng cùng một lúc trong cùng một luồng.
- Ai thấy thay đổi state: mọi code trên luồng chính dùng chung một heap, nên biến được đổi trong callback này sẽ được callback sau nhìn thấy.

4.2 Cài đặt môi trường

1. Cài Node.js bản LTS (nodejs.org hoặc nvm). LTS = Long Term Support: dòng ổn định dùng cho production.
2. Tạo thư mục dự án và chạy npm init -y. Lệnh này tạo ra package.json.
3. Cài công cụ TypeScript dưới dạng development dependency: npm i -D typescript tsx @types/node.
4. Thêm "type": "module" vào package.json để dùng được import/export và top-level await.
5. Tùy chọn: npx tsc --init để tạo tsconfig.json phục vụ kiểm tra kiểu.

Một tsconfig.json tối thiểu cho Node.js với ES Modules:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "dist"
  },
  "include": ["src"]
}
```

Các cách chạy TypeScript:

Lệnh	Tác dụng	Dùng khi
npx tsx src/index.ts	Chạy TypeScript trực tiếp	Phát triển
npx tsx watch src/index.ts	Chạy lại mỗi khi lưu file	Phát triển
npx tsc && node dist/index.js	Biên dịch ra JavaScript rồi chạy	Production / CI

Các tình huống lỗi - môi trường

Tình huống	Triệu chứng	Nguyên nhân gốc	Thời điểm
Node.js không có trong PATH	node: command not found	Trình cài đặt không thêm Node.js vào PATH, hoặc chưa mở lại terminal	Khởi động
Thiếu "type": "module"	SyntaxError: Cannot use import statement outside a module	File bị coi là CommonJS	Chạy
Sai phiên bản Node.js	Cú pháp hoặc API không được hỗ trợ	Dự án cần bản mới hơn bản đang cài	Chạy
Cài global không có quyền	Lỗi EACCES khi npm i -g	Ghi vào thư mục thuộc hệ thống	Cài đặt

Tình huống	Triệu chứng	Nguyên nhân gốc	Thời điểm
Import tương đối thiếu đuôi file (ESM đã biên dịch)	ERR_MODULE_NOT_FOUND	ESM yêu cầu tên file đầy đủ: ./notes.js	Chạy

4.3 Kiến trúc: Event Loop, V8 Engine, Non-blocking I/O

Node.js xử lý nhiều tác vụ chỉ với một luồng chính bằng cách không bao giờ ngồi chờ: nó khởi động một thao tác, đăng ký callback rồi đi tiếp. Khi thao tác xong, Event Loop sẽ chạy callback.

```
Code JS của bạn (V8, luồng chính)
       |  khởi động việc bất đồng bộ (đọc file, gọi HTTP, timer)
       v
libuv + OS  ---- làm việc chờ (thread pool / kernel của OS) ----
       |  việc đã xong
       v
Event Loop lấy callback -> chạy nó trên luồng chính
```

Các pha của Event Loop (một vòng lặp)

Pha	Chạy gì	Ví dụ điển hình
Timers	Callback của timer đã hết hạn	setTimeout, setInterval
Pending callbacks	Callback cấp hệ thống bị hoãn	Một số lỗi TCP
Poll	Sự kiện I/O mới và callback của chúng; có thể chờ ở đây	Đọc file xong, có request đến
Check	Callback của setImmediate	setImmediate
Close callbacks	Callback dọn dẹp	socket.on('close')

Giữa các callback, Node.js xử lý hết hai hàng đợi microtask: process.nextTick và callback của Promise (then, await). Microtask chạy trước khi vòng lặp đi tiếp.

```
console.log("1 sync");
setTimeout(() => console.log("4 timeout"), 0);
Promise.resolve().then(() => console.log("3 promise"));
process.nextTick(() => console.log("2 nextTick"));
```

Kết quả với ES Modules (kiểu module dùng trong khóa học này):

```
1 sync
3 promise
2 nextTick
4 timeout
```

Lưu ý: với CommonJS, thứ tự giữa promise và nextTick bị đảo (nextTick chạy trước). Với ES Modules, bản thân module chạy bên trong một promise job nên callback của promise đến trước. Đừng viết code phụ thuộc vào thứ tự này.

Blocking và non-blocking

Khía cạnh	Blocking (sync)	Non-blocking (async)
Ví dụ	readFileSync	await readFile (fs/promises)
Luồng chính khi đang chờ	Bị kẹt, không làm được gì	Rảnh để chạy các callback khác
Phong cách code	Đơn giản, tuần tự	async/await, vẫn đọc tuần tự
Phù hợp với	Script khởi động, công cụ CLI, nạp cấu hình	Server và mọi thứ phục vụ nhiều request

```
import { readFileSync } from "node:fs";
import { readFile } from "node:fs/promises";

// Blocking: không gì khác chạy cho đến khi đọc xong file
const a = readFileSync("a.txt", "utf8");

// Non-blocking: luồng chính vẫn rảnh trong khi OS đọc file
const b = await readFile("a.txt", "utf8");
```

Các tình huống lỗi - Event Loop

Tình huống	Triệu chứng	Nguyên nhân gốc	Thời điểm
Vòng lặp tốn CPU trên luồng chính	Server ngừng phản hồi trong lúc nó chạy	Event Loop không chạy được callback khác cho đến khi vòng lặp kết thúc	Runtime
Quên await trước một promise	Giá trị là object Promise; code chạy tiếp quá sớm	Hàm async trả về ngay lập tức	Runtime (âm thầm)
Promise bị reject không được xử lý	Tiến trình bị sập (mặc định ở Node.js hiện đại)	Promise bị reject không có catch / try-catch	Runtime
Đệ quy process.nextTick vô hạn	Timer và I/O không bao giờ chạy	Hàng đợi microtask được xử lý hết trước khi vòng lặp đi tiếp (starvation)	Runtime
Dùng readFileSync trong handler của request	Độ trễ tăng theo tải	Mỗi lời gọi chặn mọi request khác	Runtime khi tải cao

4.4 NPM: package.json, install, scripts

npm cài các gói từ npm registry và ghi lại chúng trong package.json. Một file điển hình:

```
{
  "name": "notekeeper",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "tsx src/index.ts",
    "watch": "tsx watch src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  },
  "dependencies": {},
  "devDependencies": {
    "@types/node": "^X.Y.Z",
    "tsx": "^X.Y.Z",
    "typescript": "^X.Y.Z"
  }
}
```

Số phiên bản cụ thể (X.Y.Z) do npm ghi vào lúc cài đặt, nên phụ thuộc vào thời điểm bạn chạy lệnh. dev chạy chương trình một lần (phù hợp công cụ CLI); watch khởi động lại chương trình mỗi lần lưu file (phù hợp server).

Trường	Mục đích
name / version	Định danh dự án (name phải viết thường, không có dấu cách)
type	"module" = ES Modules; bỏ trống = CommonJS
scripts	Các lệnh có tên, chạy bằng npm run <tên>
dependencies	Các gói cần lúc chạy (runtime)
devDependencies	Các gói chỉ cần khi phát triển/build (TypeScript, công cụ test)
engines	Khoảng phiên bản Node.js yêu cầu (tùy chọn)

Các lệnh thiết yếu

Lệnh	Tác dụng
npm install	Cài mọi thứ được liệt kê trong package.json
npm i <pkg>	Thêm một runtime dependency
npm i -D <pkg>	Thêm một development dependency
npm ci	Cài sạch, chính xác theo file lock (dùng trong CI)
npm run <script>	Chạy một script (npm start và npm test không cần "run")
npm uninstall <pkg>	Gỡ một gói
npx <tool>	Chạy binary của một gói mà không cần cài global

Semantic versioning (MAJOR.MINOR.PATCH)

Khoảng	Cho phép	Ví dụ với 1.2.3
^1.2.3	Cập nhật minor và patch	1.2.4, 1.9.0 (không phải 2.0.0)
~1.2.3	Chỉ cập nhật patch	1.2.4 (không phải 1.3.0)
1.2.3	Đúng phiên bản này	1.2.3

Bên dưới - cài đặt và phân giải module

- npm tải các gói vào thư mục node_modules bên cạnh package.json.
- package-lock.json ghi lại phiên bản chính xác của mọi gói (kể cả gói lồng bên trong), để mọi máy cài ra cùng một cây phụ thuộc.
- Khi code chạy import x from "x", Node.js tìm trong ./node_modules, rồi ../node_modules, và cứ thế đi lên cây thư mục.

Các tình huống lỗi - npm

Tình huống	Triệu chứng	Nguyên nhân gốc	Thời điểm
Không commit file lock	Mỗi máy một phiên bản khác nhau	Mỗi lần cài lại phân giải khoảng phiên bản từ đầu	Runtime
Gõ sai tên script	npm error Missing script	Script chưa được định nghĩa trong package.json	Chạy
Cài gói vào dependencies thay vì devDependencies (hoặc ngược lại)	Bản production thiếu gói hoặc chứa gói không cần thiết	Dùng sai cờ khi cài	Deploy
Thiếu hoặc xóa node_modules	Cannot find module 'x'	Chưa cài phụ thuộc sau khi git clone	Chạy
Dùng npm install trong CI	File lock bị đổi âm thầm	npm install có thể cập nhật file lock; npm ci thì không	CI

4.5 Core modules: path và fs (xử lý file)

Core module có sẵn trong Node.js. Hãy import chúng với tiền tố node: để thấy rõ đây không phải gói bên thứ ba.

path - xây dựng và phân tích đường dẫn an toàn

```
import path from "node:path";

const file = path.join("data", "notes", "todo.txt");
// data/notes/todo.txt (data\notes\todo.txt trên Windows)

path.basename(file);          // "todo.txt"
path.basename(file, ".txt");  // "todo"
path.extname(file);           // ".txt"
path.dirname(file);           // "data/notes"
path.resolve("data", "a.txt"); // đường dẫn tuyệt đối, tính từ thư mục làm việc hiện tại
```

Hàm	Trả về	Ghi chú
path.join(...parts)	Các phần được nối bằng dấu phân cách đúng	Kết quả có thể vẫn là đường dẫn tương đối
path.resolve(...parts)	Đường dẫn tuyệt đối	Phần tương đối được tính từ process.cwd()
path.basename(p, ext?)	Phần cuối (tên file)	Có thể bỏ đuôi file nếu truyền ext
path.extname(p)	Đuôi file	Có cả dấu chấm: ".txt"
path.dirname(p)	Phần thư mục	-
import.meta.dirname	Thư mục chứa file hiện tại	ES Modules, các bản Node.js gần đây; thay cho __dirname

fs - đọc và ghi file

fs có ba kiểu API cho cùng các thao tác:

Kiểu	Import	Hành vi	Dùng khi
Promises	node:fs/promises	Non-blocking, dùng với await	Lựa chọn mặc định
Callbacks	node:fs	Non-blocking, callback(err, data)	Code cũ
Sync	node:fs (...Sync)	Blocking	Chỉ dùng cho script khởi động và công cụ CLI nhỏ

```
import { mkdir, writeFile, readFile, readdir, rm } from "node:fs/promises";

await mkdir("data", { recursive: true });         // tạo thư mục nếu chưa có
await writeFile("data/note.txt", "Hello", "utf8");
const text = await readFile("data/note.txt", "utf8");
const files = await readdir("data");              // ["note.txt"]
await rm("data/note.txt");
```

Bên dưới - fs/promises hoạt động thế nào

- await readFile(...) giao việc cho libuv, và libuv chạy việc đó trên thread pool.
- Luồng chính rảnh trong lúc chờ; khi đọc xong, Promise được resolve và code của bạn chạy tiếp sau await.
- readFile nạp toàn bộ file vào bộ nhớ (một Buffer, hoặc chuỗi nếu bạn truyền "utf8"). Với file rất lớn, hãy dùng stream (createReadStream).

Xử lý lỗi file

```
try {
  await readFile("missing.txt", "utf8");
} catch (e) {
  const err = e as NodeJS.ErrnoException;
  if (err.code === "ENOENT") console.error("File not found");
  else throw err;
}
```

Mã lỗi	Ý nghĩa
ENOENT	File hoặc thư mục không tồn tại
EACCES	Không có quyền truy cập
EEXIST	File hoặc thư mục đã tồn tại
EISDIR	Mong đợi file nhưng lại gặp thư mục
ENOTEMPTY	Thư mục không rỗng (xóa mà không dùng recursive)

Các tình huống lỗi - path và fs

Tình huống	Triệu chứng	Nguyên nhân gốc	Thời điểm
Đường dẫn tương đối như "data/a.txt" khi chạy từ thư mục khác	ENOENT dù file có tồn tại	Đường dẫn tương đối tính từ process.cwd(), không phải từ file mã nguồn	Runtime
Nối dữ liệu người dùng vào đường dẫn	Có thể đọc file ngoài thư mục dự kiến	Các đoạn ../ không bị lọc (path traversal)	Runtime (bảo mật)
writeFile vào thư mục chưa tồn tại	ENOENT	writeFile không tự tạo thư mục	Runtime
readFile file nhiều GB	Tốn nhiều bộ nhớ hoặc bị sập	Toàn bộ file được nạp vào bộ nhớ	Runtime
Thiếu await khi gọi fs dạng promise	File chưa được ghi khi dòng kế tiếp chạy; unhandled rejection	Promise không được await	Runtime (âm thầm)

5. Ví dụ xuyên suốt - NoteKeeper CLI

Xuyên suốt tài liệu này, ta xây NoteKeeper, một công cụ dòng lệnh lưu mỗi ghi chú thành một file .txt.

```
notekeeper/
  package.json        <- npm: scripts, dependencies (mục 4.4)
  tsconfig.json       <- cấu hình TypeScript (mục 4.2)
  src/
    index.ts          <- điểm vào: đọc tham số dòng lệnh
    notes.ts          <- dùng path + fs/promises (mục 4.5)
  data/               <- tạo lúc chạy, mỗi ghi chú một file .txt
```

Các lệnh:

Lệnh	Kết quả
npm run dev -- add shopping "milk eggs"	Tạo data/shopping.txt
npm run dev -- list	In tiêu đề của mọi ghi chú
npm run dev -- read shopping	In nội dung ghi chú

Ví dụ này dùng lại mọi phần của bài học: môi trường, kiến trúc non-blocking, npm scripts và các module path/fs. Nó cũng sẽ được dùng lại ở phần Demo và Case Study.

6. Demo - Từng bước: từ script ngây thơ đến NoteKeeper

Bước 1 - Tạo dự án

```
mkdir notekeeper && cd notekeeper
npm init -y
npm i -D typescript tsx @types/node
npm pkg set type=module
npm pkg set scripts.dev="tsx src/index.ts"
npm pkg set scripts.watch="tsx watch src/index.ts"
mkdir src
```

NoteKeeper là công cụ dòng lệnh: chạy một lệnh rồi thoát, nên ta dùng dev (chạy một lần). Nếu sau này bạn biến nó thành server, hãy dùng npm run watch để chương trình tự khởi động lại mỗi khi bạn lưu file.

Bước 2 - notes.ts: path + fs/promises

```
// src/notes.ts
import { mkdir, readdir, readFile, writeFile } from "node:fs/promises";
import path from "node:path";

const DATA_DIR = path.join(import.meta.dirname, "..", "data");

function notePath(title: string): string {
  // basename bỏ mọi phần thư mục, regex thay các ký tự lạ
  const safe = path.basename(title).replace(/[^a-z0-9_-]/gi, "-");
  // từ chối tiêu đề rỗng hoặc vô nghĩa như "" hoặc "???"
  if (!safe || /^-+$/.test(safe)) throw new Error("Invalid note title");
  return path.join(DATA_DIR, `${safe}.txt`);
}

export async function addNote(title: string, body: string): Promise<void> {
  await mkdir(DATA_DIR, { recursive: true });
  await writeFile(notePath(title), body, "utf8");
}

export async function listNotes(): Promise<string[]> {
  await mkdir(DATA_DIR, { recursive: true });
  const files = await readdir(DATA_DIR);
  return files
    .filter((f) => path.extname(f) === ".txt")
    .map((f) => path.basename(f, ".txt"));
}

export async function readNote(title: string): Promise<string> {
  return readFile(notePath(title), "utf8");
}
```

Vì sao tốt hơn? Đường dẫn được tạo bằng path.join (portable), không phụ thuộc thư mục hiện tại (import.meta.dirname), dữ liệu người dùng được làm sạch và bị từ chối khi vô nghĩa (rỗng hoặc chỉ có ký hiệu), và mọi lời gọi đều non-blocking.

Bước 3 - index.ts: đọc tham số dòng lệnh

```
// src/index.ts
import { addNote, listNotes, readNote } from "./notes.js";

const [command, title, ...rest] = process.argv.slice(2);

try {
  if (command === "add" && title) {
    await addNote(title, rest.join(" "));
    console.log(`Saved note "${title}"`);
  } else if (command === "list") {
    console.log((await listNotes()).join("\n") || "(no notes)");
  } else if (command === "read" && title) {
    console.log(await readNote(title));
  } else {
    console.log("Usage: add <title> <text> | list | read <title>");
  }
} catch (e) {
  const err = e as NodeJS.ErrnoException;
  console.error(err.code === "ENOENT" ? "Note not found" : err.message);
  process.exitCode = 1;
}
```

Lưu ý: dòng import kết thúc bằng .js dù file thật là notes.ts. Đây là yêu cầu của ES Modules (xem các tình huống lỗi ở mục 4.2); cả tsx và tsc đều hiểu cách viết này.

Bước 4 - Chạy thử

```
npm run dev -- add shopping milk eggs
# -> Saved note "shopping"
npm run dev -- list
# -> shopping
npm run dev -- read shopping
# -> milk eggs
npm run dev -- read unknown
# -> Note not found
npm run dev -- add "???" test
# -> Invalid note title
```

Bước 5 - Chứng minh nó là non-blocking

Đọc nhiều ghi chú cùng lúc bằng Promise.all giúp các lần đọc chồng lên nhau thay vì chạy lần lượt:

```
const titles = await listNotes();
const bodies = await Promise.all(titles.map((t) => readNote(t)));
console.log(bodies.length, "notes loaded");
```

Tình huống lỗi - đường dẫn tương đối phụ thuộc thư mục hiện tại:

```
// SAI: chỉ chạy được khi bạn khởi động chương trình từ thư mục gốc của dự án
await readFile("data/shopping.txt", "utf8");

// Nếu chạy từ thư mục khác:
// Error: ENOENT: no such file or directory, open 'data/shopping.txt'
```

Trace lúc chạy cho lệnh "add shopping milk eggs":

```
1. node khởi động V8, nạp index.js, chạy code cấp cao nhất
2. process.argv.slice(2) -> ["add", "shopping", "milk", "eggs"]
3. command = "add", title = "shopping", rest = ["milk", "eggs"]
4. addNote("shopping", "milk eggs") bắt đầu
5. mkdir(...) được giao cho libuv; luồng chính dừng ở "await" nhưng không bị chặn
6. mkdir xong -> Promise resolve -> writeFile(...) được giao cho libuv
7. writeFile xong -> console.log("Saved note ...")
8. Event Loop hết việc -> tiến trình thoát với mã 0
```

7. Tự kiểm chứng

Chạy script sau để tự kiểm chứng thứ tự của Event Loop. Lưu thành order.ts (với "type": "module" trong package.json):

```
// order.ts
console.log("1 sync");
setTimeout(() => console.log("4 timeout"), 0);
Promise.resolve().then(() => console.log("3 promise"));
process.nextTick(() => console.log("2 nextTick"));
```

Chạy: npx tsx order.ts

Kết quả mong đợi (ES Modules):

```
1 sync
3 promise
2 nextTick
4 timeout
```

Thử nghiệm:

- Đổi tên file thành order.cjs, bỏ các chú thích kiểu và chạy node order.cjs. Thứ tự của 2 và 3 thay đổi - vì sao?
- Thêm dòng setImmediate(() => console.log("5 immediate")). Nó luôn được in trước hay sau timeout? Hãy chạy vài lần để xem.

8. Case Study

[Case minh họa - không phải một công ty có thật]

Bối cảnh: Một nhóm xây dựng dịch vụ nội bộ phục vụ các mẫu báo cáo HTML. Phiên bản đầu đọc mẫu từ đĩa ở mỗi request bằng readFileSync và tạo đường dẫn bằng chuỗi như "templates\\" + name.

Các vấn đề xuất hiện:

1. Lập trình viên dùng macOS thấy lỗi "file not found" vì đường dẫn dùng dấu gạch chéo ngược.
2. Vào giờ cao điểm, dịch vụ chậm với tất cả mọi người, kể cả những request không cần đọc mẫu nào.
3. Một đợt rà soát bảo mật phát hiện ?name=../../.env có thể đọc file ngoài thư mục templates.

Cách khắc phục, dùng những gì tài liệu này đã dạy:

1. Tạo đường dẫn bằng path.join(import.meta.dirname, "templates", path.basename(name)) - portable và an toàn.
2. Thay readFileSync bằng await readFile từ node:fs/promises để Event Loop luôn rảnh; có thể cache mẫu trong bộ nhớ sau lần đọc đầu tiên.
3. Xử lý ENOENT và trả về 404 thay vì làm sập dịch vụ.
4. Commit package-lock.json vào git và dùng npm ci trong pipeline build để mọi máy có cùng phụ thuộc.

Kết quả: cùng một đoạn code chạy được trên mọi hệ điều hành, một lần đọc đĩa chậm không còn làm đứng các request khác, và lỗ hổng path traversal đã được bịt.

9. Lỗi thường gặp - Tra cứu nhanh

Event Loop và async

Quên await:

```
// SAI - content là một Promise, không phải chuỗi
const content = readFile("a.txt", "utf8");
console.log(content); // Promise { <pending> }

// ĐÚNG
const content = await readFile("a.txt", "utf8");
```

Lệnh blocking trong server:

```
// SAI bên trong handler của request
const data = readFileSync("big.json", "utf8");

// ĐÚNG
const data = await readFile("big.json", "utf8");
```

npm

```
# SAI - commit các gói đã cài
git add node_modules

# ĐÚNG - bỏ qua nó, commit package.json + package-lock.json
echo node_modules >> .gitignore
```

path và fs

```
// SAI - phụ thuộc OS, nối chuỗi
const file = "data\\" + name;

// ĐÚNG
const file = path.join("data", name);
```

```
// SAI - phụ thuộc nơi bạn khởi động chương trình
await readFile("data/a.txt", "utf8");

// ĐÚNG - tính từ file mã nguồn này
await readFile(path.join(import.meta.dirname, "data", "a.txt"), "utf8");
```

Module

```
// SAI trong ESM đã biên dịch - thiếu đuôi file
import { addNote } from "./notes";

// ĐÚNG
import { addNote } from "./notes.js";
```

Tổng hợp nhanh

Lỗi	Triệu chứng	Cách sửa một dòng
Thiếu await	Promise { <pending> } hoặc code chạy quá sớm	Thêm await
Lệnh sync trong server	Server đứng khi tải cao	Dùng fs/promises
Đường dẫn tự nối	Hỏng trên hệ điều hành khác	Dùng path.join
Đường dẫn tương đối theo cwd	ENOENT khi chạy từ thư mục khác	Dùng import.meta.dirname
node_modules trong git	Repository rất nặng	Thêm vào .gitignore
Thiếu "type": "module"	Cannot use import statement outside a module	Khai báo trong package.json
Không xử lý lỗi	Tiến trình bị sập	try/catch và kiểm tra err.code

10. Best Practices

10.1 Ưu tiên API async; giữ luồng chính luôn rảnh

Mặc định hãy dùng fs/promises với async/await. Chỉ dùng các hàm *Sync cho code khởi động (nạp cấu hình) hoặc công cụ CLI nhỏ chạy một lần.

10.2 Khi nào KHÔNG nên dùng - theo từng khái niệm

Khái niệm	KHÔNG dùng khi	Hậu quả	Dấu hiệu	Lựa chọn tốt hơn
Luồng chính của Node.js	Việc nặng về CPU (xử lý ảnh, tính toán lớn)	Mọi request phải chờ	CPU kẹt ở 100% trên một core, phản hồi chậm	worker_threads hoặc một dịch vụ riêng
Hàm fs đồng bộ	Trong handler của request hoặc vòng lặp phục vụ người dùng	Server đứng khi tải cao	Độ trễ tăng theo lượng truy cập	fs/promises
readFile	File rất lớn	Tốn nhiều bộ nhớ	Bộ nhớ tăng đột biến	Stream (createReadStream)
Thêm dependency mới	Chỉ vài dòng code là làm được	Bề mặt tấn công lớn hơn, phải cập nhật nhiều hơn	Gói chỉ dùng cho một hàm nhỏ	Tự viết hàm, hoặc dùng core module
Khoảng phiên bản ^	Cần build tái lập mà không có file lock	Bị nâng phiên bản bất ngờ	Build khác nhau giữa các ngày	Commit file lock và dùng npm ci

10.3 Đánh đổi - callback, promise và lệnh sync

Khía cạnh	Sync (readFileSync)	Promises (fs/promises)	Callbacks (fs.readFile)
Ưu điểm	Code đơn giản nhất	Non-blocking, dễ đọc với await, xử lý lỗi dễ	Chạy ở mọi nơi, không cần khái niệm thêm
Nhược điểm	Chặn luồng chính	Phải nhớ await và xử lý lỗi	Code lồng nhau (callback hell), xử lý lỗi khó hơn
Dùng khi	Khởi động, công cụ nhỏ	Mặc định trong code mới	Code cũ bạn đang bảo trì
Rủi ro	Làm đứng server	Unhandled rejection nếu không await	Quên tham số lỗi

10.4 Điều nhiều người hiểu sai nhất

Hiểu lầm phổ biến: "Node.js chạy một luồng nên chậm và không thể phục vụ nhiều người dùng."

Thực tế: Chỉ JavaScript của bạn chạy trên một luồng. Việc chờ đĩa và mạng do OS và libuv đảm nhận, nên một tiến trình Node.js có thể giữ hàng nghìn kết nối mở. Điều làm Node.js gặp vấn đề không phải là "một luồng" mà là chặn luồng đó - bằng lệnh sync hoặc vòng lặp CPU dài. Học Node.js nghĩa là học cách giữ Event Loop luôn rảnh.

11. Bài tập tự luyện (chạy code, đọc output)

Nhóm: Môi trường và npm

Bài 1 (Dễ): Cài Node.js bản LTS, chạy node -v và npm -v, rồi ghi lại các phiên bản. Tạo hello.ts và chạy bằng npx tsx.

Bài 2 (Dễ): Tạo dự án mới bằng npm init -y. Thêm các script dev, build và start. Chạy npm run dev.

Bài 3 (Trung bình): Cài một gói tiện ích nhỏ bằng npm i, và một gói liên quan đến kiểu bằng npm i -D. Xem package.json và package-lock.json. Mỗi gói nằm ở mục nào? Xóa node_modules và khôi phục bằng npm ci.

Nhóm: Event Loop

Bài 4 (Dễ): Chạy script order.ts ở mục 7. Giải thích từng dòng của output.

Bài 5 (Trung bình): Dự đoán output rồi chạy thử:

```
console.log("A");
setTimeout(() => console.log("B"), 0);
(async () => {
  console.log("C");
  await null;
  console.log("D");
})();
console.log("E");
```

Bài 6 (Khó): Viết một vòng lặp chạy trong 3 giây (while (Date.now() < end) {}). Đặt một setTimeout 100 ms trước vòng lặp đó. Khi nào callback của timer chạy? Vì sao nó không chạy sau 100 ms?

Nhóm: path

Bài 7 (Dễ): Với đường dẫn /users/intern/docs/report.final.pdf, in ra basename, extname, dirname và basename không có đuôi.

Bài 8 (Trung bình): Viết hàm safeJoin(baseDir, userInput) trả về đường dẫn đã nối, nhưng ném lỗi nếu kết quả nằm ngoài baseDir. Thử với ../secret.txt và với một thư mục anh em có tên bắt đầu giống baseDir (ví dụ baseDir = /app/data và đích = /app/data-secret/file.txt). Gợi ý: dùng path.resolve, rồi kiểm tra kết quả bằng path.relative (kết quả không được bắt đầu bằng .. và không được là đường dẫn tuyệt đối), hoặc so sánh với baseDir + path.sep. Hãy suy nghĩ vì sao startsWith(baseDir) đơn thuần là chưa đủ.

Nhóm: fs

Bài 9 (Dễ): Viết script tạo thư mục out, ghi 3 file .txt vào đó và in tên các file bằng fs/promises.

Bài 10 (Trung bình): Đọc một file không tồn tại. Bắt lỗi và chỉ in thông báo thân thiện khi err.code là ENOENT; với các lỗi khác thì ném lại.

Bài 11 (Trung bình): So sánh readFileSync và readFile với Promise.all trên 5 file, mỗi file 50 MB. Đo thời gian bằng console.time. Cách nào nhanh hơn, và vì sao kết quả có thể khác với trường hợp chỉ một file?

Tổng hợp

Bài 12 (Khó): Mở rộng NoteKeeper với các lệnh delete <title> và append <title> <text>. Yêu cầu: chỉ dùng fs/promises, xử lý ENOENT với thông báo rõ ràng, giữ mọi đường dẫn bên trong thư mục data, và thêm script build biên dịch bằng tsc cùng script start chạy dist/index.js.

Bài 13 (Trung bình): Viết test tự động cho phần xử lý tiêu đề của NoteKeeper bằng các module có sẵn node:test và node:assert (không cần cài thêm gói). Export notePath từ notes.ts, rồi thêm các test case cho: một tiêu đề bình thường ("shopping"), một tiêu đề rỗng, một tiêu đề chỉ gồm ký hiệu ("???") và một tiêu đề cố thoát ra khỏi thư mục ("../../x"). Với mỗi trường hợp, hãy quyết định hành vi đúng là gì (một đường dẫn an toàn hay một lỗi), viết kỳ vọng trước, rồi chạy test bằng node --test (dùng tsx nếu bạn giữ file TypeScript). Trường hợp nào khiến bạn bất ngờ, và bạn đã sửa gì trong code?

12. Recap

Những gì đã học

Khái niệm	Ý chính	Từ khóa / lệnh
Node.js	JavaScript runtime xây trên V8 + libuv	node, V8, libuv
Môi trường	Cài LTS, khởi tạo dự án, thêm công cụ TypeScript	node -v, npm init, tsx
Event Loop	Một luồng chính chạy callback khi việc đã sẵn sàng	timers, poll, check, microtasks
Non-blocking I/O	Khởi động việc, đi tiếp, quay lại khi xong	async/await, Promise

Khái niệm	Ý chính	Từ khóa / lệnh
npm	Cài gói và chạy script; khóa phiên bản	package.json, npm i, npm ci, npm run
path	Tạo và phân tích đường dẫn portable	join, resolve, basename, extname
fs	Đọc và ghi file; ưu tiên API promise	readFile, writeFile, readdir, mkdir

Mental model - nhà hàng

- Luồng chính = một người phục vụ. Anh ta nhận order, chuyển cho bếp (libuv / OS) rồi lập tức phục vụ bàn kế tiếp.
- Event Loop = quy tắc của người phục vụ: khi bếp bấm chuông, mang món đó ra.
- Blocking = người phục vụ đứng trong bếp chờ một món; mọi bàn khác phải đợi.
- npm = danh sách nhà cung cấp: chính xác nguyên liệu (gói) nào và phiên bản nào.

13. Góc nhìn Fresher

Bài này dạy nền tảng của Node.js. Fresher không cần biết mọi chi tiết bên trong - điều quan trọng là tư duy: "không chặn Event Loop" và "để công cụ quản lý đường dẫn và phụ thuộc".

Những gì cần tập trung

1. Async là mặc định: dùng await với fs/promises; hiểu vì sao lệnh sync nguy hiểm trong server.
2. Đọc kỹ thông báo lỗi: ENOENT, EACCES, ERR_MODULE_NOT_FOUND cho biết chính xác điều gì sai.
3. Commit package.json và package-lock.json, không bao giờ commit node_modules.
4. Dùng path.join và import.meta.dirname thay cho đường dẫn tự nối chuỗi.
5. Không đưa dữ liệu người dùng trực tiếp vào đường dẫn file.

Sai lầm Fresher hay mắc

- Quên await và in ra Promise { <pending> }.
- Dùng readFileSync khắp nơi vì "nó cứ chạy được".
- Cài mọi thứ global (npm i -g) thay vì theo từng dự án.
- Bỏ qua lỗi - không có try/catch quanh các thao tác file.
- Chép gói vào thư mục dự án thay vì cài bằng npm.

Nguyên tắc Fresher: một file = một trách nhiệm; một hàm = một việc; một await = một bước rõ ràng. Làm được 3 điều này, code Node.js của bạn đã tốt hơn nhiều người mới học.